

AI in the Sciences and Engineering

Final Project Report

Giovanni Guidarini 25-945-684
gguidarini@ethz.ch

Fall Semester 2025

Contents

1	Visualizing Loss Landscapes: PINNs vs. Data-Driven	2
1.1	Task 1: Data Generation	2
1.2	Task 2: Implementation and Training	2
1.2.1	Physics-Informed Neural Network	2
1.2.2	Data-Driven Model	2
1.2.3	Training & Results	2
1.3	Task 3: Loss Landscape Visualization	3
2	Training an FNO to approximate a dynamical system	4
2.1	Task 1: One-to-One Training	4
2.2	Task 2: Testing on Different Resolutions	4
2.3	Task 3: All2All Training	4
2.3.1	Training on all time snapshot pairs	4
2.3.2	Testing at final time	4
2.3.3	Testing at multiple time steps	5
2.4	Task 4: Finetuning	5
2.4.1	Zero-Shot test	5
2.4.2	Finetuning	5
2.4.3	Bonus: Train a model from scratch	5
3	Geometry-Aware Operator Transformer	6
3.1	Task 1: Establishing a Baseline	6
3.2	Task 2: Random Sampling & Dynamic Radius Strategy	6
3.2.1	Random Sampling	6
3.2.2	Dynamic Radius	6
A	Task 1 Figures	7
B	Task 2 Figures	12
C	Task 3 Figures	13

1 Visualizing Loss Landscapes: PINNs vs. Data-Driven

1.1 Task 1: Data Generation

Synthetic data consisting of forcing-term and solution pairs for the Poisson equation on the unit square (with homogeneous Dirichlet boundary conditions) was generated using the closed-form expressions provided in the task sheet. If needed, diversity across samples can be ensured by varying the random seed used to generate the coefficients $a_{ij} \sim \mathcal{N}(0, 1)$. Three examples of source-solution fields at increasing complexities are shown in figure 1.

1.2 Task 2: Implementation and Training

1.2.1 Physics-Informed Neural Network

To ensure consistency in the comparison between Data-Driven and Physics-informed approaches, both the PINN and Data-Driven model are built upon the same MLP class. This features 4 hidden layers with a width of 128 neurons. The total loss is a composite of the boundary term and the PDE residual:

$$\mathcal{L}_{\text{PINN}}(\theta) = \frac{1}{N_f} \sum |-\Delta \hat{u}_\theta - f|^2 + \lambda \mathcal{L}_{\text{BC}}$$

The Loss function directly incorporates physics knowledge by using the PDE residual. Automatic differentiation is exploited in calculating the Laplacian $\Delta \hat{u}_\theta$. The constant λ was set to 1.

1.2.2 Data-Driven Model

The Data-Driven model is constructed as a standard MLP which aims to approximate the solution $u(x, y)$ for a single source field via regression.

The loss function is the standard Mean Squared Error (MSE) between the network prediction and the exact solution:

$$\mathcal{L}_{\text{Data}} = \frac{1}{N_d} \sum |\hat{u}_\theta(x) - u_{\text{exact}}(x)|^2$$

Unlike the PINN, this approach is agnostic to the underlying differential operator and relies solely on the density of the training data. The Data-Driven model serves as the baseline for demonstrating that the architecture has enough expressivity to capture the complexity of the solution.

1.2.3 Training & Results

In order to directly observe the spectral bias of PINNs, both models are trained with a hybrid optimization strategy across all frequency levels $K \in \{1, 4, 16\}$. First, Adam optimizer is used for 4000 epochs; this allows the model to reach a sufficiently good area of the loss landscape. Afterwards, L-BFGS optimizer with a strong Wolfe line search comes into play; this in order to rapidly converge to the local minimum with high precision.

Target values for the Data-Driven model were scaled by a factor of 100 to stabilize gradient magnitudes during backpropagation. The experiments were conducted for three complexity levels: low ($K = 1$), medium ($K = 4$), and high ($K = 16$). The relative L_2 errors for both models are reported in table 1. Predictions of both models at all complexity levels are shown in figure 2.

Complexity (K)	Relative L_2 Error (%)	
	PINN	Data-Driven
1	0.13	0.01
4	2.47	0.05
16	197.66	2.01

Table 1: Comparison of Relative L_2 errors between PINN and Data-Driven models.

The accuracy displayed by the Data-Driven model at $K=16$ clearly shows that the network has enough expressivity to fully capture the complexity of the solution; even when high frequency eigenfunctions are included.

In figure 2, it can be observed that the PINN mostly struggles to approximate the solution at the boundary. In order to address this issue, multiple boundary loss weights λ were experimented with. However, with the adopted training procedure, the best performing one was $\lambda = 1$. An interpretation of this behaviour is that increasing the boundary loss weight leads to a degradation in the PDE residual loss minimization; which further worsens the overall performance of the PINN. Also, increasing the

number of boundary collocation points used during training didn't lead to any significant improvement in performance.

The difficulty in optimizing the PINN at high complexity is captured in figure 3. These loss evolution graphs show that for $K = 16$ the optimization problem is ill-conditioned; making it extremely hard for the optimizer to minimize the physics-informed loss function.

1.3 Task 3: Loss Landscape Visualization

In order to visualize the loss landscape of the models, the filter normalization technique proposed in [1] was adopted. The directions for the perturbations were sampled randomly as described in the paper. The original code provided by the authors was adapted to work with the PINN and Data-Driven models implemented for this task.

Figure 5 shows the comparison between the loss landscapes of the PINN and Data-Driven models at all complexity levels. The plots clearly show that data-driven models exhibit a smooth and well behaved loss landscape. In contrast, as the complexity increases, the PINN's landscape becomes increasingly steep, culminating for $K = 16$ in an extremely narrow well.

The visualization of the loss landscape at $K = 16$ reveals that the model is trapped in a very deep local minimum, with massively steep walls surrounding it. Even though this landscape doesn't display roughness and multiple local minima, this extreme sharpness is still a symptom of an ill-conditioned optimization problem. In fact, as discussed in [2], there are some directions in which the landscape is extremely steep while in others it's almost flat. This causes optimizers to struggle in finding a descent direction that effectively reduces the loss. This extreme sharpness arises from the fact that the Physics-Informed loss function incorporates a squared laplacian term, which is known to have high condition number when dealing with high frequency components. This leads to large gradient magnitudes which make the optimization path very erratic and prone to overshooting, thus resulting in challenging optimization.

When interpreting these results, it's important to consider that the loss landscape visualizations are based on 2D slices of the high-dimensional parameter space. While these slices provide valuable insights into the local geometry of the loss landscape; they may not fully capture the complexity of the entire landscape. For this reason, a second visualization was attempted, where the loss value is perturbed along the first two dominant Hessian eigenvectors, as suggested in [3]. Plotting using the largest hessian eigenvectors as directions, for the PINNs produced loss landscapes so steep that in order to appreciate the plots, the maximum plotted quote had to be raised by three orders of magnitude, (Figure 6). The PINN for $K = 16$, also exhibits a "rough" landscape with multiple local minima. Moreover, the computation of the largest Hessian eigenvectors and eigenvalues confirmed the extreme sharpness of the PINN loss landscape at high complexity. Results are summarized in table 2.

Complexity (K)	Largest Hessian Eigenvalue	
	PINN	Data-Driven
1	8.997423e+02	8.559248e+03
4	4.608101e+03	4.487791e+02
16	6.104677e+04	3.084370e+02

Table 2: Largest Hessian Eigenvalues for PINN and Data-Driven models at varying complexities.

The results clearly show that as the complexity increases, the largest Hessian eigenvalue of the PINN grows significantly, indicating an increasingly sharp loss landscape. In contrast, the Data-Driven model's largest Hessian eigenvalue decreases with complexity, suggesting a smoother loss landscape. One notable aspect is that at $K = 1$, the Data-Driven model has a larger largest Hessian eigenvalue than the PINN, yet it remains trainable and achieves good accuracy. This highlights that while the largest Hessian eigenvalue is a useful indicator of local loss landscape sharpness, it does not solely determine trainability, since its significance is restricted to a neighbourhood of the local minimum.

2 Training an FNO to approximate a dynamical system

2.1 Task 1: One-to-One Training

For one-to-one training of the FNO on the unknown dynamical system, a standard implementation of the FNO architecture was adopted. The network features 3 Fourier layers with 16 modes each and width of 64. The training was performed on the full dataset `data_train_128.npy` for 2000 epochs using Adam optimizer with ReduceLROnPlateau scheduler.

The trained model achieved an average relative L_2 error of 1.54% on the test set `data_test_128.npy`.

2.2 Task 2: Testing on Different Resolutions

For testing the trained FNO on different resolutions, the model was evaluated on datasets `data_test_{s}.npy`, where $s \in \{32, 64, 96, 128\}$. The observed relative L_2 errors are summarized in table 3.

Resolution	Relative L_2 Error (%)	
	16 modes	12 modes
32	16.07	7.38
64	4.05	3.04
96	2.18	1.60
128	1.54	1.16

Table 3: FNO Performance across different resolutions.

The results clearly show that even though the model can be evaluated at different resolutions, its performance degrades as the resolution diverges from the training resolution of 128. This suggests that while FNOs have some degree of resolution invariance, their accuracy is still influenced by the training resolution.

The sharp degradation in performance at $N = 32$ (16.07% error) is attributed to breaking the Nyquist-Shannon barrier. With the FNO configured to preserve 16 Fourier modes, a resolution of $N = 32$ corresponds to the critical sampling limit ($k_{\text{Nyq}} = N/2 = 16$). This results in significant aliasing artifacts, where high-frequency content learned during training ($N_{\text{train}} = 128$) folds back into lower frequencies upon discretization, corrupting the prediction. At $N = 64$, the Nyquist frequency increases to 32, resolving the 16 modes without aliasing.

Having observed the aliasing effects at the lowest resolution, a second FNO was trained, featuring a reduced number of Fourier modes equal to 12. The same training procedure as before was followed. The performance of this new model is summarized in table 3.

Reducing the number of modes to 12 mitigated aliasing effects and significantly reduced the degradation in performance at the lowest resolution of 32. Moreover, the overall performance across all resolutions improved, this proved to be a fortunate case where reducing the number of modes actually enhanced generalization. This might come down to the fact that the bigger model with 16 modes needs more training to fully exploit its expressivity, while the smaller model is easier to optimize to its full potential. However, for more complex tasks, like All2All training, reducing the number of modes proved to be detrimental, as shown in the next section.

2.3 Task 3: All2All Training

2.3.1 Training on all time snapshot pairs

For the all-to-all training of the FNO, the architecture was modified by incorporating time-conditional normalization layers after each Fourier layer and by including time as an additional input channel.

The training was conducted on all time snapshots pairs $(u(t_a), u(t_b))$ with $t_a, t_b \in \{0, 0.25, \dots, 1.0\}$, $t_a < t_b$ extracted from the dataset `data_train_128.npy`. The width of the FNO was kept the same as in the one-to-one training. Two different models were trained: one with 16 Fourier modes and another with 12 Fourier modes. The training was performed for 200 epochs using Adam optimizer with ReduceLROnPlateau scheduler.

2.3.2 Testing at final time

The trained all-to-all FNO models were evaluated on the test set `data_test_128.npy` by predicting the solution at final time $t = 1.0$ given the initial condition at $t = 0.0$. The models achieved an average relative L_2 error of 1.8480% (16 modes) and 2.6926% (12 modes), which is slightly higher than their respective one-to-one trained counterparts.

This might come down to the fact that by using all time snapshot pairs in training, the models have to learn a wider range of dynamics, which could lead to a trade-off in accuracy for specific time intervals. If the goal is to optimize performance at a specific time while maintaining the ability to generalize across time, a more complex architecture might be necessary. This hypothesis is corroborated by the fact that the more expressive model of the two achieved better performance on all time snapshots.

However, the error increase is relatively small with respect to one-to-one. This indicates that the all-to-all training approach is effective in capturing the underlying dynamics of the system while providing flexibility in time prediction.

2.3.3 Testing at multiple time steps

To evaluate the all-to-all trained FNO models' performance across multiple time steps, the models were tested on the same test set `data_test_128.npy` by predicting solutions at intermediate time steps $t \in \{0.25, 0.5, 0.75, 1.0\}$ given the initial condition at $t = 0.0$.

The observed relative L_2 errors at different time steps are summarized in table 4.

Time Step	Relative L_2 Error (%)	
	16 modes	12 modes
0.25	1.5738	2.1899
0.5	1.4889	2.2466
0.75	1.5232	2.1731
1.0	1.8480	2.6926

Table 4: Relative L_2 errors at varying time steps.

The results indicate that the FNO with 16 Fourier modes consistently outperforms the one with 12 modes across all time steps. This suggests that the additional expressivity provided by the higher number of modes is beneficial for capturing the dynamics of the system over time.

Moreover, both models exhibit relatively stable performance across the different time steps, with only a slight increase in error at the final time step $t = 1.0$. This stability indicates that the all-to-all training approach effectively equips the FNO models to generalize across time, maintaining accuracy throughout the evolution of the dynamical system. The increased error at $t = 1.0$ could be attributed to the shortage of training data pairs covering the one second time interval between $t = 0.0$ and $t = 1.0$.

2.4 Task 4: Finetuning

2.4.1 Zero-Shot test

For this task, only the all-to-all trained FNO with 16 Fourier modes was considered, as it demonstrated superior performance in previous evaluations.

The model obtained in task 3 was directly evaluated on an unseen dataset `data_test_unknown_128.npy`. The focus was on predictions at final time $t = 1.0$ given the initial condition at $t = 0.0$. The model achieved an average relative L_2 error of 9.4975% on this unseen dataset. The performance on an unseen distribution is significantly worse compared to the test set from the known distribution.

2.4.2 Finetuning

The model was then finetuned on the unseen dataset `data_finetime_unknown_128.npy` in all-to-all fashion for 100 epochs using Adam optimizer with ReduceLROnPlateau scheduler.

The finetuning improved the model's performance, reducing the average relative L_2 error on all time steps. Results are shown in table 5 and figure 9.

2.4.3 Bonus: Train a model from scratch

To determine whether knowledge acquired on the pre-training dataset `data_train_128.npy` is useful for the unseen distribution, a new FNO with 16 Fourier modes was trained from scratch with the same all-to-all procedure on the unseen dataset `data_finetime_unknown_128.npy` for 100 epochs.

Time Step	Relative L_2 Error (%)		
	Zero-Shot	Finetuned	From Scratch
0.25	7.2598	5.2644	10.9357
0.5	8.4500	5.8335	10.8692
0.75	9.0877	6.3953	11.8286
1.0	9.4975	7.0463	12.7794

Table 5: FNO Performance across different resolutions.

This model performed worse than both the finetuned model and the zero-shot model on all time steps, as shown in table 5. This demonstrated that pre-training on a related dataset provides valuable prior knowledge that enhances learning on new, unseen distributions.

3 Geometry-Aware Operator Transformer

3.1 Task 1: Establishing a Baseline

The performance baseline using the official GAOT implementation [4] was obtained with a model tailored for the available hardware. This model operates on a fixed latent grid resolution of 48×48 , resulting in a total of 2,304 tokens. Other parameters were reduced leading to a model with a total parameter count of 1912913, which is roughly half the size of the original GAOT model for the elasticity problem. The model was trained for 200 epochs on the dataset `Elasticity.nc` with Adam optimizer. The model achieved a test relative L_1 error of 2.04% on the elasticity problem.

The total training time was one hour and six minutes on an Nvidia RTX 3070 laptop GPU.

3.2 Task 2: Random Sampling & Dynamic Radius Strategy

3.2.1 Random Sampling

Random downsampling of tokens from the underlying point cloud was implemented by modifying the graph building pipeline. The number of tokens to sample was set to 256, a significantly smaller amount compared to the baseline model; in order to compensate for the reduced number of tokens, the fixed radius was increased from 0.033 to 0.25. The rest of the architecture remained the same as the baseline. This model was trained for 200 epochs on the same dataset and with the same training procedure as before. The model achieved a test relative L_1 error of 7.27% on the elasticity problem and required 59 minutes to train on the same hardware.

Other Random Sampling techniques were experimented with, such as randomly sampling coordinates in the domain and random coordinate sampling with quasi-random Sobol sequences. However, these methods didn't lead to any significant improvement in performance compared to naive random sampling of tokens from the point cloud. So the latter was chosen as the representative random sampling strategy for this task.

3.2.2 Dynamic Radius

In order to enhance the random sampling strategy, a dynamic radius approach was implemented. This method adjusts the radius used for neighborhood graph construction. The dynamic radius r_l for each token y_l is computed based on the distance to its k -th nearest neighbor:

$$r_l = \alpha \cdot d(y_l, y_{(k)})$$

where $d(y_l, y_{(k)})$ is the distance from token y_l to its k -th nearest neighbor, and α is a scaling factor. For this implementation, k was set to 8 and α to 1.5.

The rest of the architecture and training procedure remained unchanged from the random sampling model.

This model achieved a test relative L_1 error of 8.92% on the elasticity problem and required 1 hour to train.

The dynamic radius strategy led to a slight decrease in performance compared to the static radius approach. This could be due to the fact that the dynamic radius parameters were not optimally tuned, even though the used values were the best found through experimentation. However, the performance remained similar to the static radius model, and also the mean computed radius across tokens was close to the static radius value of 0.25.

The significant drop in performance obtained by random sampling strategies compared to the baseline model can be attributed to the significant reduction in the number of tokens which the model processes. This led to the inability of the model to capture the finer dynamics of the differential operator. However, the model was able to achieve reasonable accuracy despite the aggressive downsampling, capturing the overall behavior of the operator, as depicted in figures 10 and 11.

A Task 1 Figures

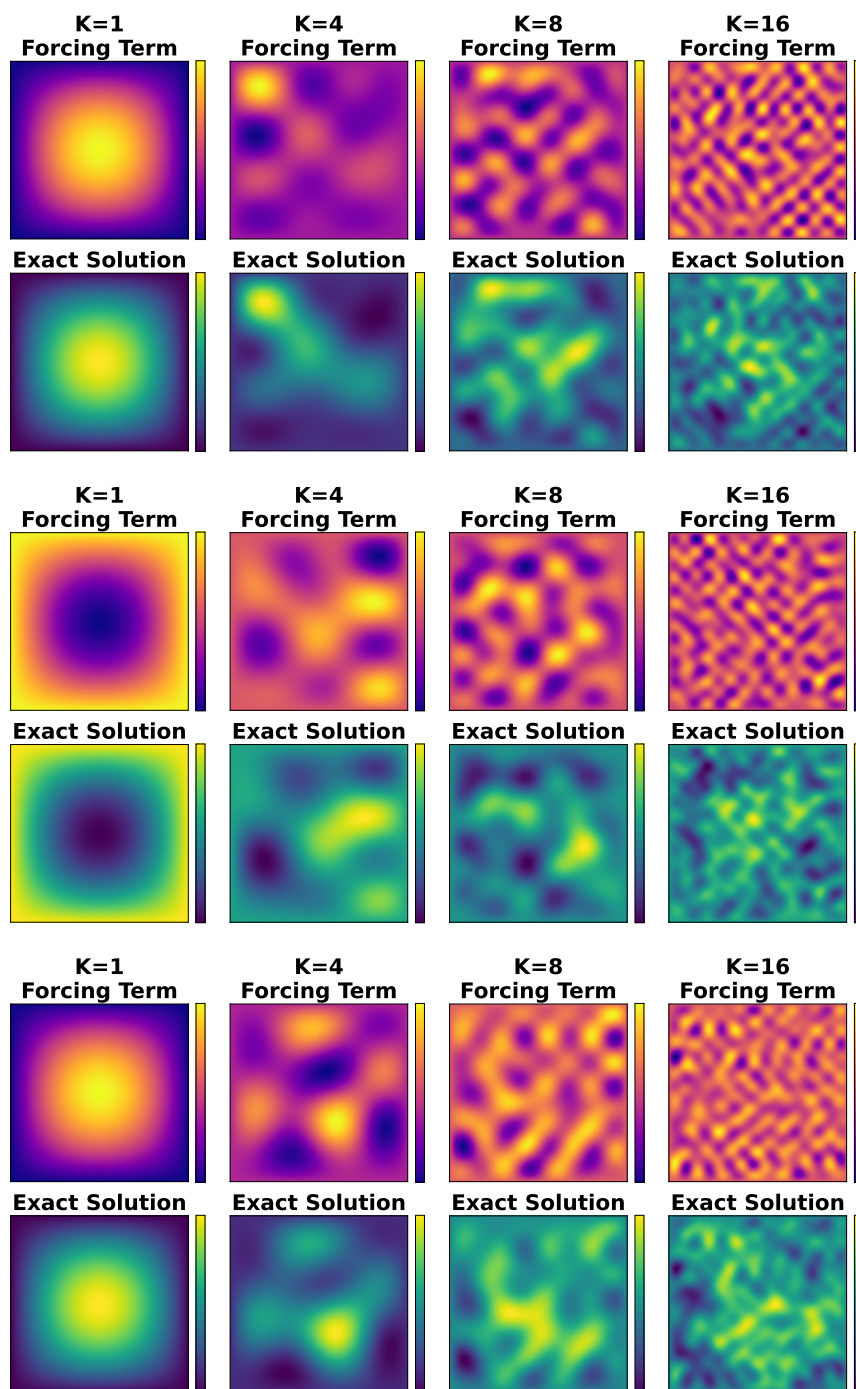
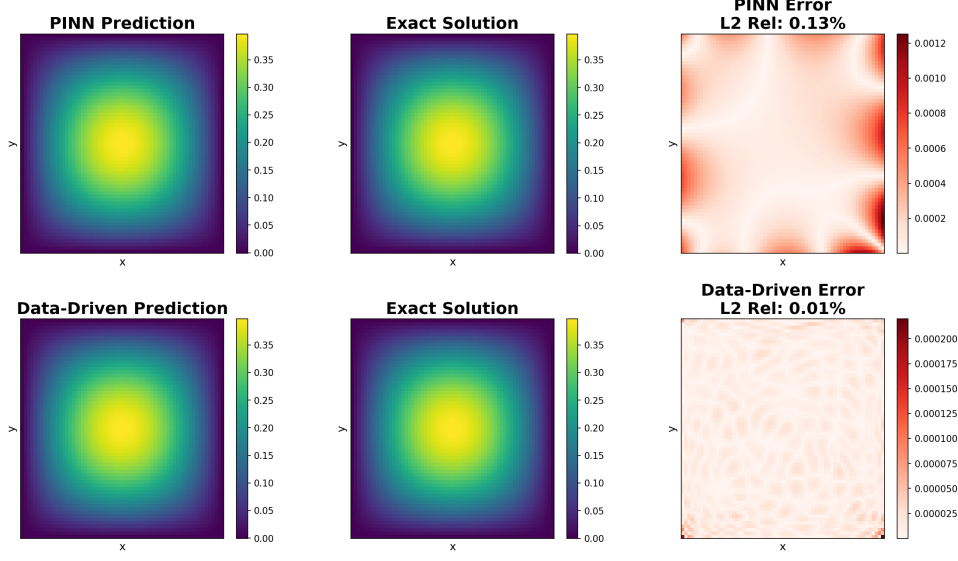
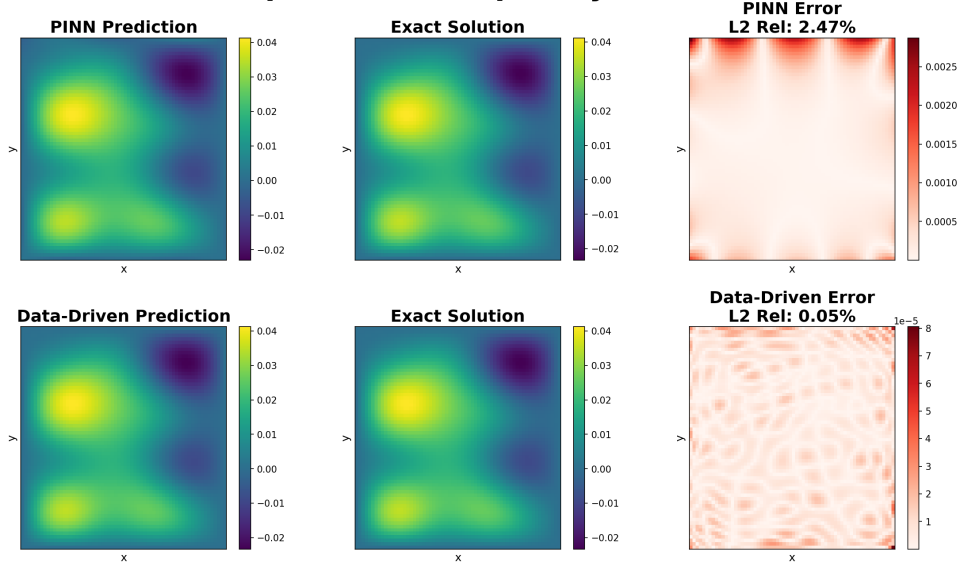


Figure 1: Three examples of source field f and the corresponding solution field u for $K = 1, 4, 8, 16$.

Model Comparison - Complexity Low (K=1)



Model Comparison - Complexity Medium (K=4)



Model Comparison - Complexity High (K=16)

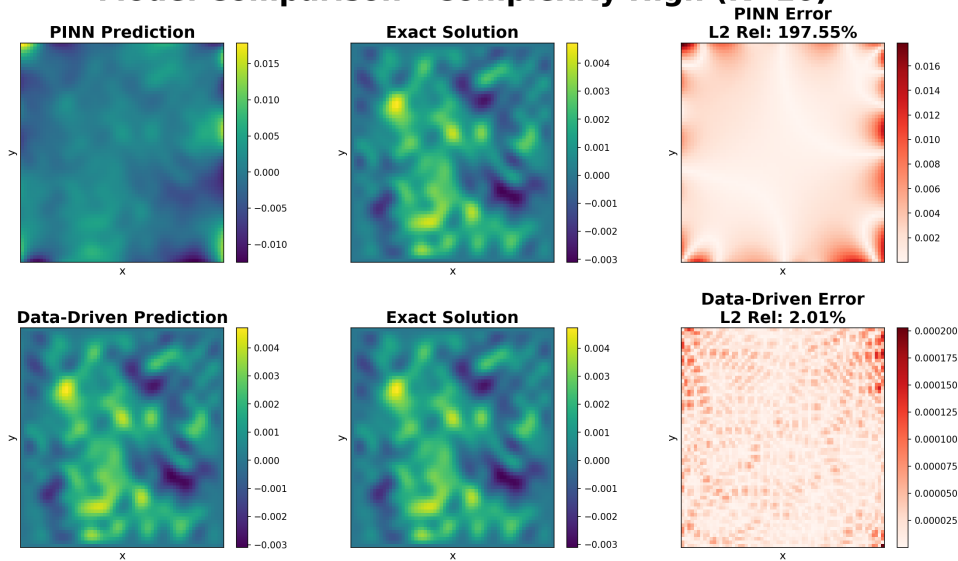


Figure 2: Predictions of PINN and Data-Driven models at increasing complexities, compared to the exact solution.

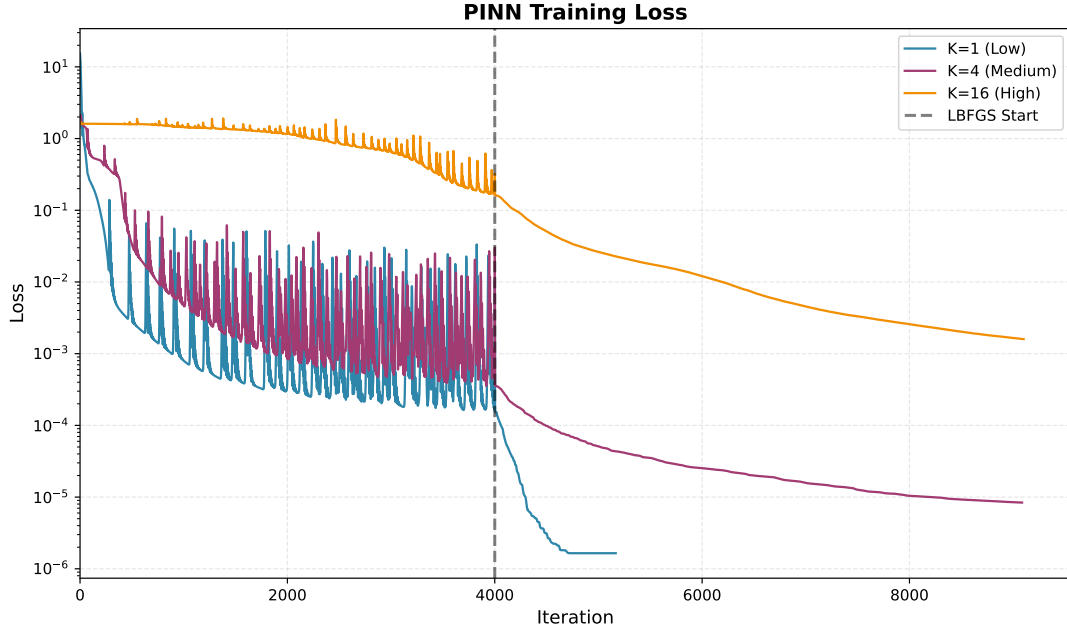


Figure 3: Training loss curve of the PINN at three problem complexities.

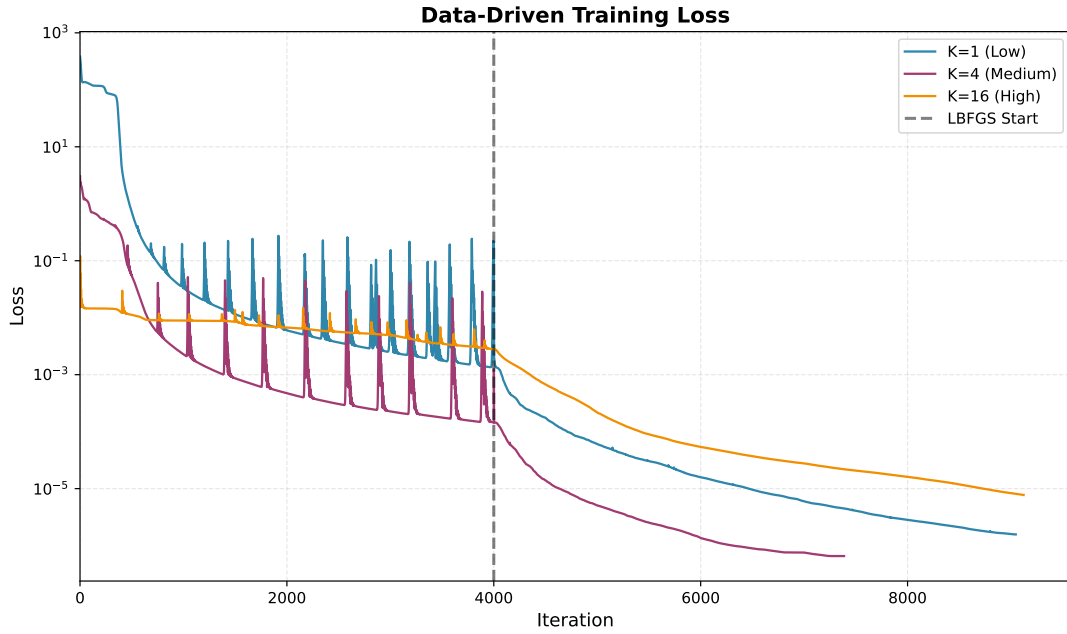


Figure 4: Training loss curve of the Data-Driven model at three problem complexities.

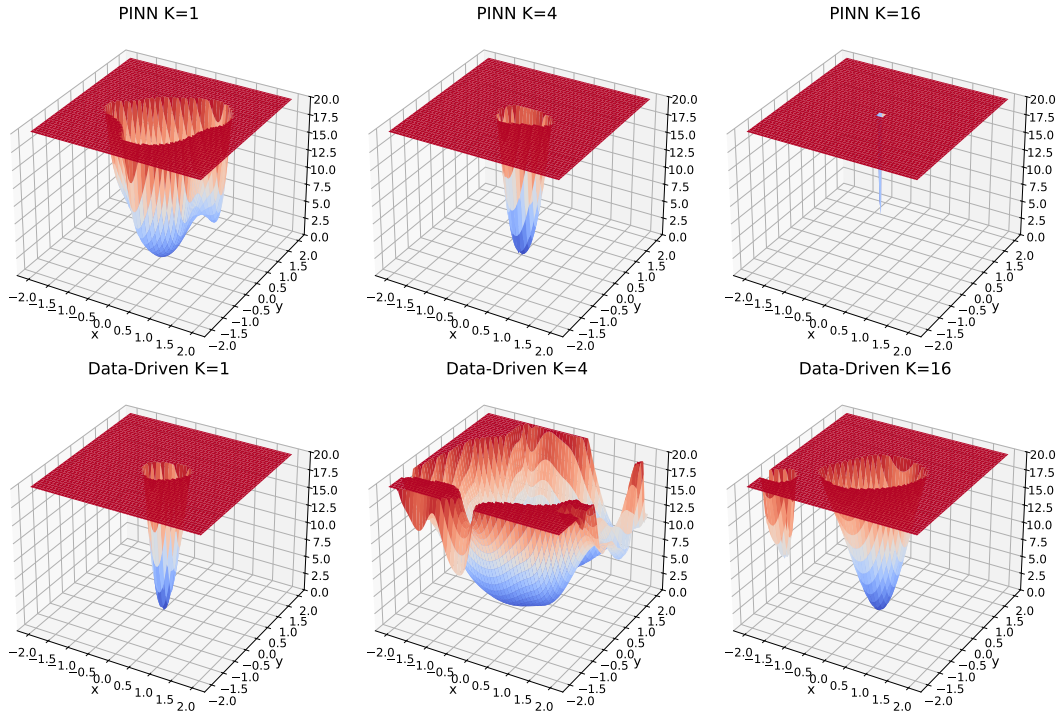
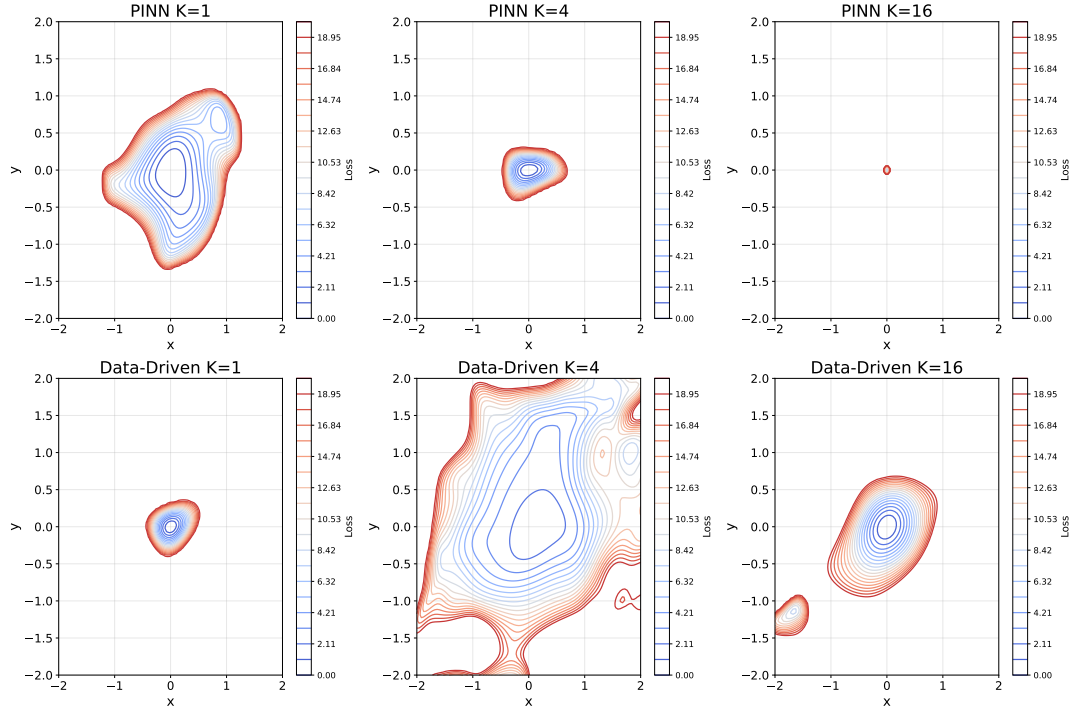


Figure 5: Loss landscape comparison between PINN and Data-Driven models at increasing complexities, generated using random perturbation directions.

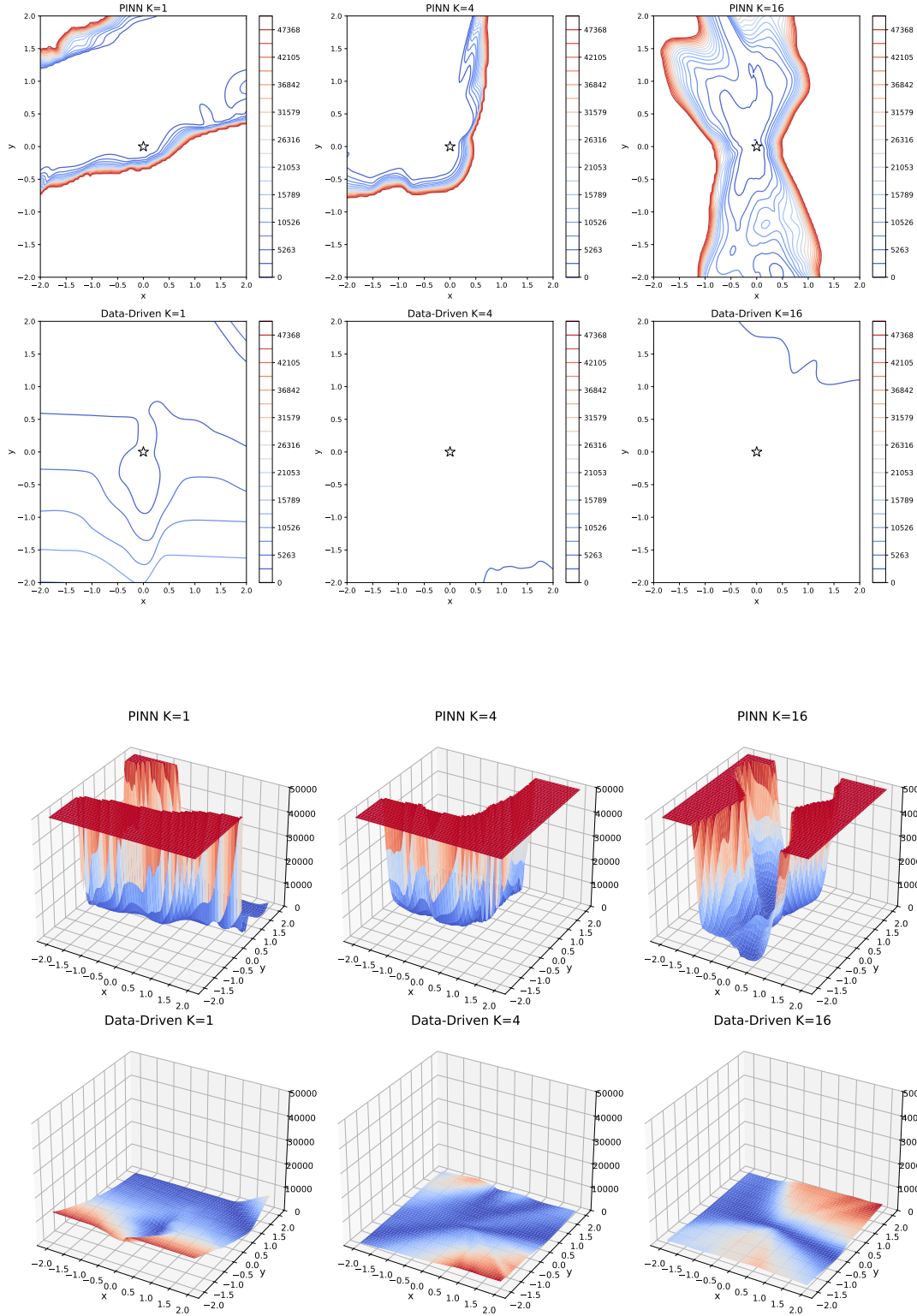


Figure 6: Loss landscape comparison between PINN and Data-Driven models at increasing complexities, generated using the first two dominant Hessian eigenvectors as perturbation directions. It can be observed that all PINN landscapes are extremely sharp, especially for the PINN at $K=16$, where the local minimum is so steep that the Data-Driven loss landscape appears to be flat in comparison. For the Data-Driven model trained on $K = 1$ this visualization highlights the locality of the information provided by the Hessian eigenvalues. In fact, away from the converged minimum, the loss landscape appears much flatter than all PINN landscapes, even though some of these had smaller Hessian spectral radii.

B Task 2 Figures

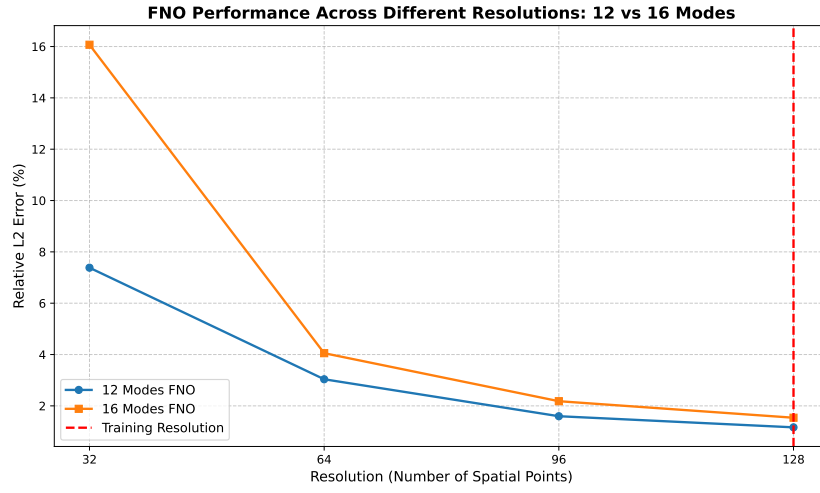


Figure 7: FNO Performance across different resolutions, the red dotted line indicates the training resolution.

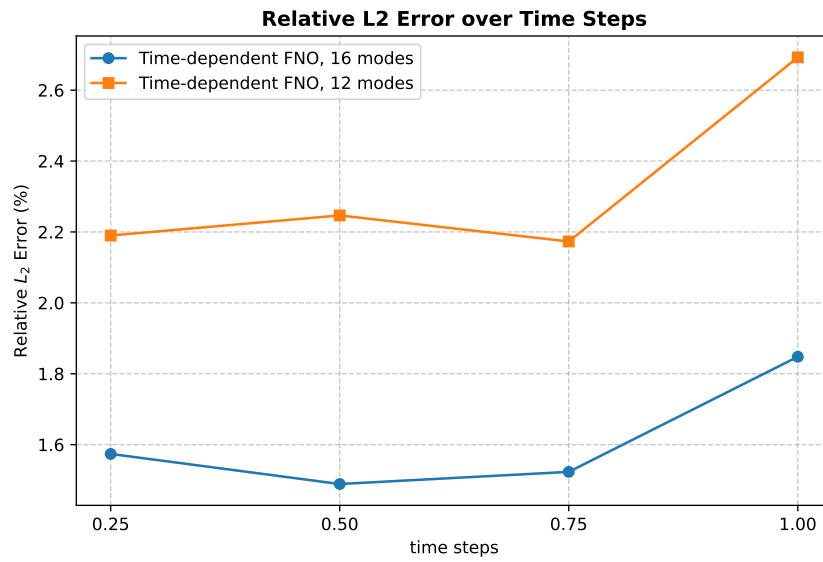


Figure 8: Visual comparison of 16-modes FNO vs 12-modes FNO predictions at different time steps.

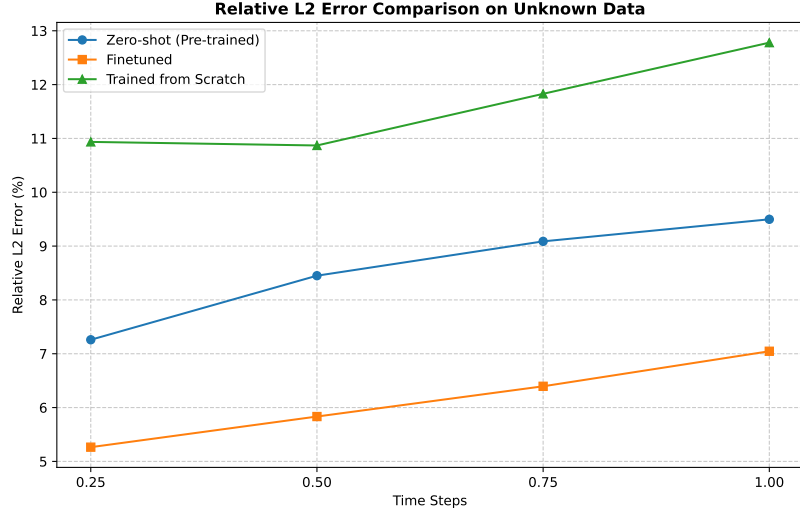


Figure 9: Visual comparison of Zero-Shot, Finetuned and From-Scratch 16-modes FNO predictions at final time step.

C Task 3 Figures

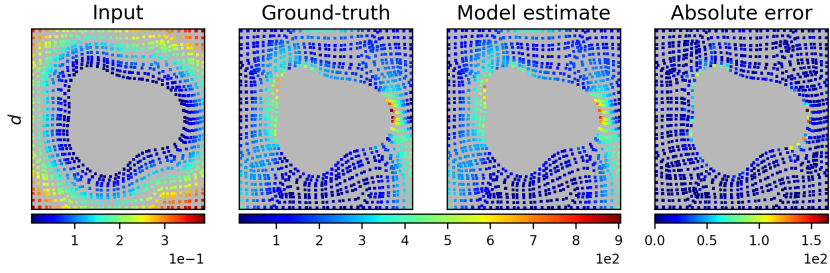


Figure 10: GAOT predictions with random sampling and static radius.

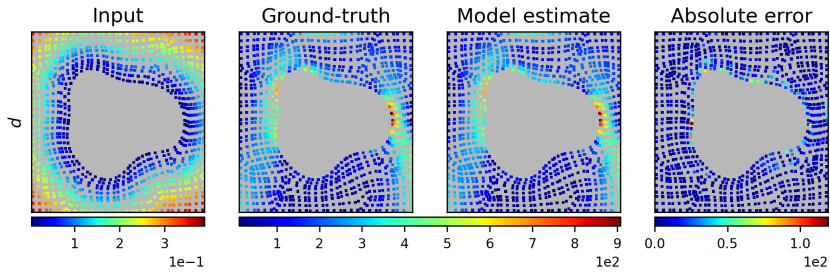


Figure 11: GAOT predictions with random sampling and dynamic radius.

References

- [1] Hao Li et al. *Visualizing the Loss Landscape of Neural Nets*. 2018. arXiv: 1712.09913 [cs.LG]. URL: <https://arxiv.org/abs/1712.09913>.
- [2] Aditi S. Krishnapriyan et al. *Characterizing possible failure modes in physics-informed neural networks*. 2021. arXiv: 2109.01050 [cs.LG]. URL: <https://arxiv.org/abs/2109.01050>.
- [3] Zhewei Yao et al. *PyHessian: Neural Networks Through the Lens of the Hessian*. 2020. arXiv: 1912.07145 [cs.LG]. URL: <https://arxiv.org/abs/1912.07145>.
- [4] Shizheng Wen et al. *Geometry Aware Operator Transformer as an Efficient and Accurate Neural Surrogate for PDEs on Arbitrary Domains*. 2026. arXiv: 2505.18781 [cs.LG]. URL: <https://arxiv.org/abs/2505.18781>.